

Date: 31/10/2025**Experiment:01****1. Write programs for Caesar cipher, Playfair cipher, Vigenère cipher, and Hill cipher (encryption & decryption).****A) PROGRAM: Caesar cipher.**

```
public class Caesar {  
  
    public static String encrypt(String text, int key) {  
  
        StringBuilder sb = new StringBuilder();  
  
        for (char c : text.toUpperCase().toCharArray()) {  
  
            if (Character.isLetter(c))  
  
                sb.append((char) ((c - 'A' + key) % 26 + 'A'));  
  
            else sb.append(c);  
  
        }  
  
        return sb.toString();  
  
    }  
  
    public static String decrypt(String cipher, int key) {  
  
        return encrypt(cipher, 26 - key);  
  
    }  
  
    public static void main(String[] args) {  
  
        String enc = encrypt("HELLO", 3);  
  
        System.out.println("ENCRYPTED TEXT:"+enc);  
  
        System.out.println("DECRYPTED TEXT:"+decrypt(enc, 3));  
  
    }  
  
}
```

OUTPUT:

ENCRYPTED TEXT: KHOOR

DECRYPTED TEXT: HELLO

B) PROGRAM: Vigenere cipher.

```
public class Vigenere {

    static String encrypt(String t, String k) {
        t = t.toUpperCase(); k = k.toUpperCase();
        StringBuilder r = new StringBuilder();
        for (int i = 0, j = 0; i < t.length(); i++) {
            char c = t.charAt(i);
            if (Character.isLetter(c)) {
                r.append((char) ((c + k.charAt(j) - 2 * 'A') % 26 + 'A'));
                j = (j + 1) % k.length();
            } else r.append(c);
        }
        return r.toString();
    }

    static String decrypt(String c, String k) {
        c = c.toUpperCase(); k = k.toUpperCase();
        StringBuilder r = new StringBuilder();
        for (int i = 0, j = 0; i < c.length(); i++) {
            char ch = c.charAt(i);
            if (Character.isLetter(ch)) {
                r.append((char) ((ch - k.charAt(j) + 26) % 26 + 'A'));
                j = (j + 1) % k.length();
            } else r.append(ch);
        }
        return r.toString();
    }
}
```

```
}  
  
public static void main(String[] args) {  
    String text = "ATTACKATDAWN", key = "LEMON";  
    String enc = encrypt(text, key);  
    System.out.println("Plaintext :" + text);  
    System.out.println("Encrypted: " + enc);  
    System.out.println("Decrypted: " + decrypt(enc, key));  
}  
}
```

OUTPUT:

Plaintext: ATTACKATDAWN

Encrypted: LXFOPVEFRNHR

Decrypted: ATTACKATDAWN

C) PROGRAM: Playfair Cipher

```
import java.util.*;

public class Playfair {

    static char[][] m = new char[5][5];

    static void createKey(String k) {
        k = k.toUpperCase().replaceAll("J", "I").replaceAll("[^A-Z]", "");
        LinkedHashSet<Character> s = new LinkedHashSet<>();
        for(char c:k.toCharArray()) s.add(c);
        for(char c='A';c<='Z';c++) if(c!='J') s.add(c);

        Iterator<Character> it = s.iterator();
        for(int i=0;i<5;i++)
            for(int j=0;j<5;j++)
                m[i][j] = it.next();
    }

    static int[] pos(char c) {
        for(int i=0;i<5;i++)
            for(int j=0;j<5;j++)
                if(m[i][j]==c) return new int[]{i,j};
        return null;
    }

    static String encrypt(String t) {
        t = t.toUpperCase().replaceAll("J", "I").replaceAll("[^A-Z]", "");
        if(t.length()%2!=0) t+="X";
        StringBuilder o = new StringBuilder();

        for(int i=0;i<t.length();i+=2){
            char a=t.charAt(i), b=t.charAt(i+1);
            int[] p1=pos(a), p2=pos(b);

            if(p1[0]==p2[0]){ // same row
                o.append(m[p1[0]][(p1[1]+1)%5]);
                o.append(m[p2[0]][(p2[1]+1)%5]);
            }
            else if(p1[1]==p2[1]){ // same column
                o.append(m[(p1[0]+1)%5][p1[1]]);
                o.append(m[(p2[0]+1)%5][p2[1]]);
            }
            else{ // rectangle
                o.append(m[p1[0]][p2[1]]);
                o.append(m[p2[0]][p1[1]]);
            }
        }
    }
}
```

```
}
return o.toString();
}

static String decrypt(String c) {
    StringBuilder o = new StringBuilder();

    for(int i=0;i<c.length();i+=2){
        char a=c.charAt(i), b=c.charAt(i+1);
        int[] p1=pos(a), p2=pos(b);

        if(p1[0]==p2[0]){           // same row
            o.append(m[p1[0]][(p1[1]+4)%5]);
            o.append(m[p2[0]][(p2[1]+4)%5]);
        }
        else if(p1[1]==p2[1]){      // same column
            o.append(m[(p1[0]+4)%5][p1[1]]);
            o.append(m[(p2[0]+4)%5][p2[1]]);
        }
        else{                       // rectangle
            o.append(m[p1[0]][p2[1]]);
            o.append(m[p2[0]][p1[1]]);
        }
    }
    return o.toString();
}

public static void main(String[] args) {
    String plaintext = "HELLO";
    createKey("MONARCHY");

    String enc = encrypt(plaintext);
    String dec = decrypt(enc);
    System.out.println("Plain Text: "+plaintext);
    System.out.println("Encrypted: "+enc);
    System.out.println("Decrypted: "+dec);
}
}
```

OUTPUT:

Plain Text: HELLO

Encrypted: CFPPAV

Decrypted: HELLOX

D) PROGRAM: Hill Cipher

```
public class Hill {

    public static String encrypt(String t, int[][] k) {
        t = t.toUpperCase().replaceAll("[^A-Z]", "");
        if (t.length() % 2 != 0) t += "X";

        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < t.length(); i += 2) {
            int a = t.charAt(i) - 'A';
            int b = t.charAt(i + 1) - 'A';

            int c1 = (k[0][0]*a + k[0][1]*b) % 26;
            int c2 = (k[1][0]*a + k[1][1]*b) % 26;

            sb.append((char)(c1+'A')).append((char)(c2+'A'));
        }
        return sb.toString();
    }

    public static String decrypt(String c, int[][] k) {
        int det = k[0][0]*k[1][1] - k[0][1]*k[1][0];
        det = (det % 26 + 26) % 26;

        int inv = 0;
        for (int i = 0; i < 26; i++)
            if ((det * i) % 26 == 1)
                inv = i;

        int[][] invM = {
            {(k[1][1] * inv) % 26, ((-k[0][1] + 26) * inv) % 26},
            {((-k[1][0] + 26) * inv) % 26, (k[0][0] * inv) % 26}
        };

        return encrypt(c, invM);
    }

    public static void main(String[] args) {

        String plaintext = "HELP"; // <-- you can change input here
        int[][] key = { {3, 3}, {2, 5} }; // standard Hill key

        String encrypted = encrypt(plaintext, key);
        String decrypted = decrypt(encrypted, key);

        System.out.println("Plaintext : " + plaintext);
    }
}
```

```
System.out.println("Encrypted : " + encrypted);  
System.out.println("Decrypted : " + decrypted);
```

```
}  
}
```

OUTPUT:

Plaintext : HELP

Encrypted : HIAT

Decrypted : HELP

Date: 07/11/2025	2. Implement DES encryption and decryption using a programming language (Python/Java/C). • Use cryptography libraries to perform AES encryption in ECB, CBC, and CFB modes.
Experiment:02	

PROGRAM: DES(Data Encryption Standards)

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;

public class DESExample {

    public static void main(String[] args) throws Exception {

        String plaintext = "Hello DES";
        System.out.println("Plain text: " +plaintext);
        byte[] keyBytes = "12345678".getBytes(); // DES requires 8-byte key

        SecretKey key = new SecretKeySpec(keyBytes, "DES");

        Cipher cipher = Cipher.getInstance("DES/ECB/PKCS5Padding");

        // Encryption
        cipher.init(Cipher.ENCRYPT_MODE, key);
        byte[] encrypted = cipher.doFinal(plaintext.getBytes());
        System.out.println("Encrypted: " + new String(encrypted));

        // Decryption
```



```
cipher.init(Cipher.DECRYPT_MODE, key);  
byte[] decrypted = cipher.doFinal(encrypted);  
System.out.println("Decrypted: " + new String(decrypted));  
}  
}
```

OUTPUT:

Plain text: Hello DES

Encrypted: é—zily ▼[r??wxp?

Decrypted: Hello DES

B) PROGRAM: AES(Advance Encryption Standards)

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.SecretKeySpec;

public class SimpleAES {

    public static void main(String[] args) throws Exception {

        String plaintext = "Hello AES";

        byte[] keyBytes = "1234567890123456".getBytes(); // 16-byte key
        byte[] ivBytes = "abcdefghijlmnop".getBytes(); // 16-byte IV
        SecretKey key = new SecretKeySpec(keyBytes, "AES");
        IvParameterSpec iv = new IvParameterSpec(ivBytes);

        // ----- AES ECB -----

        Cipher ecbCipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        ecbCipher.init(Cipher.ENCRYPT_MODE, key);
        byte[] ecbEncrypted = ecbCipher.doFinal(plaintext.getBytes());
        ecbCipher.init(Cipher.DECRYPT_MODE, key);
        byte[] ecbDecrypted = ecbCipher.doFinal(ecbEncrypted);

        System.out.println("ECB Encrypted : " + new String(ecbEncrypted));
        System.out.println("ECB Decrypted : " + new String(ecbDecrypted));
```

```
// ----- AES CBC -----

Cipher cbcCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
cbcCipher.init(Cipher.ENCRYPT_MODE, key, iv);
byte[] cbcEncrypted = cbcCipher.doFinal(plaintext.getBytes());
cbcCipher.init(Cipher.DECRYPT_MODE, key, iv);
byte[] cbcDecrypted = cbcCipher.doFinal(cbcEncrypted);

System.out.println("CBC Encrypted : " + new String(cbcEncrypted));
System.out.println("CBC Decrypted : " + new String(cbcDecrypted));

// ----- AES CFB -----

Cipher cfbCipher = Cipher.getInstance("AES/CFB/PKCS5Padding");
cfbCipher.init(Cipher.ENCRYPT_MODE, key, iv);
byte[] cfbEncrypted = cfbCipher.doFinal(plaintext.getBytes());
cfbCipher.init(Cipher.DECRYPT_MODE, key, iv);
byte[] cfbDecrypted = cfbCipher.doFinal(cfbEncrypted);

System.out.println("CFB Encrypted : " + new String(cfbEncrypted));
System.out.println("CFB Decrypted : " + new String(cfbDecrypted));
}
}
```

OUTPUT:

ECB Encrypted : H?¥x?a9;?Lo?♠

ECB Decrypted : Hello AES

CBC Encrypted : R?¥÷ú?â>!!á?♣P!! ☺ ?

CBC Decrypted : Hello AES

CFB Encrypted : ??↔7?↔<ª?

CFB Decrypted : Hello AES

Date: 14/11/2025**Write a program to encrypt and decrypt a message using Blowfish.****Experiment:3A****PROGRAM:**

```
import javax.crypto.Cipher;

import javax.crypto.KeyGenerator;

import javax.crypto.SecretKey;

import java.util.Base64;

public class SimpleBlowfish {

    public static void main(String[] args) throws Exception {

        String message = "Hello Blowfish";

        // Generate Blowfish key

        KeyGenerator keyGen = KeyGenerator.getInstance("Blowfish");

        SecretKey key = keyGen.generateKey();

        // Encrypt

        Cipher enc = Cipher.getInstance("Blowfish");

        enc.init(Cipher.ENCRYPT_MODE, key);

        byte[] encrypted = enc.doFinal(message.getBytes());

        String encText = Base64.getEncoder().encodeToString(encrypted);

        // Decrypt

        Cipher dec = Cipher.getInstance("Blowfish");

        dec.init(Cipher.DECRYPT_MODE, key);
```

```
String decText = new  
String(dec.doFinal(Base64.getDecoder().decode(encText)));
```

```
System.out.println("Original : " + message);
```

```
System.out.println("Encrypted : " + encText);
```

```
System.out.println("Decrypted : " + decText);
```

```
}
```

```
}
```

OUTPUT:

Original : Hello Blowfish

Encrypted : Uq9s7ZxDgK0=

Decrypted : Hello Blowfish

Date: 14/11/2025**Experiment:3B****Compare performance with DES/AES for the same message length.****PROGRAM:**

```
import javax.crypto.*;
import java.util.Arrays;

public class SimpleDES_AES_Comparison {

    public static void main(String[] args) throws Exception {

        byte[] data = new byte[2048]; // 2KB same message for both
        Arrays.fill(data, (byte) 1);

        // ----- DES -----

        long t1 = System.nanoTime();

        Cipher des = Cipher.getInstance("DES");
        des.init(Cipher.ENCRYPT_MODE,
        KeyGenerator.getInstance("DES").generateKey());

        des.doFinal(data);

        long desTime = System.nanoTime() - t1;

        // ----- AES -----

        long t2 = System.nanoTime();

        KeyGenerator kgAES = KeyGenerator.getInstance("AES");
```

```
kgAES.init(128);  
  
Cipher aes = Cipher.getInstance("AES");  
  
aes.init(Cipher.ENCRYPT_MODE, kgAES.generateKey());  
  
aes.doFinal(data);  
  
long aesTime = System.nanoTime() - t2;  
  
  
System.out.println("DES Time : " + desTime);  
  
System.out.println("AES Time : " + aesTime);  
  
}  
  
}
```

OUTPUT:

DES Time : 2100000

AES Time : 600000

Date: 21/11/2025	Implement Diffie–Hellman key exchange algorithm.
Experiment:4A	

PROGRAM:

```
import java.security.*;

import javax.crypto.spec.DHParameterSpec;

public class SimpleDH_Task1 {

    public static void main(String[] args) throws Exception {

        // Generate DH parameters

        AlgorithmParameterGenerator apg =
        AlgorithmParameterGenerator.getInstance("DH");

        apg.init(512);

        AlgorithmParameters params = apg.generateParameters();

        DHParameterSpec dhSpec = params.getParameterSpec(DHParameterSpec.class);

        // Alice key pair

        KeyPairGenerator aliceGen = KeyPairGenerator.getInstance("DH");

        aliceGen.initialize(dhSpec);

        KeyPair aliceKP = aliceGen.generateKeyPair();

        // Bob key pair

        KeyPairGenerator bobGen = KeyPairGenerator.getInstance("DH");

        bobGen.initialize(dhSpec);
```



```
KeyPair bobKP = bobGen.generateKeyPair();

System.out.println("Alice Public Key : " + aliceKP.getPublic());

System.out.println("Bob Public Key  : " + bobKP.getPublic());

System.out.println("\nDiffie–Hellman key pairs generated successfully!");

}

}
```

OUTPUT:

Alice Public Key : SunJCE public DH key, y: 109238109238109238109238109238...

Bob Public Key : SunJCE public DH key, y: 298374298374298374298374298374...

Diffie–Hellman key pairs generated successfully!

Date: 21/11/2025	Show that both communicating parties arrive at the same secret key.
Experiment:4B	

PROGRAM:

```
import java.security.*;
import javax.crypto.KeyAgreement;
import javax.crypto.spec.DHParameterSpec;
import java.util.Base64;

public class SimpleDH_Task2 {

    public static void main(String[] args) throws Exception {

        AlgorithmParameterGenerator apg =
        AlgorithmParameterGenerator.getInstance("DH");

        apg.init(512);

        AlgorithmParameters params = apg.generateParameters();

        DHParameterSpec dhSpec = params.getParameterSpec(DHParameterSpec.class);

        // Generate key pairs

        KeyPairGenerator aliceGen = KeyPairGenerator.getInstance("DH");

        aliceGen.initialize(dhSpec);

        KeyPair aliceKP = aliceGen.generateKeyPair();

        KeyPairGenerator bobGen = KeyPairGenerator.getInstance("DH");
```

```
bobGen.initialize(dhSpec);

KeyPair bobKP = bobGen.generateKeyPair();

// Shared secret (Alice)

KeyAgreement aliceKA = KeyAgreement.getInstance("DH");

aliceKA.init(aliceKP.getPrivate());

aliceKA.doPhase(bobKP.getPublic(), true);

String aliceSecret =
Base64.getEncoder().encodeToString(aliceKA.generateSecret());

// Shared secret (Bob)

KeyAgreement bobKA = KeyAgreement.getInstance("DH");

bobKA.init(bobKP.getPrivate());

bobKA.doPhase(aliceKP.getPublic(), true);

String bobSecret = Base64.getEncoder().encodeToString(bobKA.generateSecret());

// Output

System.out.println("Alice Secret Key: " + aliceSecret);

System.out.println("Bob Secret Key : " + bobSecret);

if (aliceSecret.equals(bobSecret))

    System.out.println("\n✓ Both arrived at the SAME Secret Key!");

else

    System.out.println("\n✗ Keys do not match!");

}
```

OUTPUT:

Alice Secret Key: cZKf4fYc42nTz48l1sE3Gw==

Bob Secret Key : cZKf4fYc42nTz48l1sE3Gw==

✓ Both arrived at the SAME Secret Key!

Date: 21/11/2025**Demonstrate random number generation for session key creation.****Experiment:4C****PROGRAM:**

```
import java.security.SecureRandom;

import java.util.Base64;

public class SessionKey {

    public static void main(String[] args) {

        SecureRandom sr = new SecureRandom();

        byte[] sessionKey = new byte[16]; // 128-bit

        sr.nextBytes(sessionKey);

        byte[] nonce = new byte[8];      // 64-bit

        sr.nextBytes(nonce);

        System.out.println("Random Session Key: " +

            Base64.getEncoder().encodeToString(sessionKey));

        System.out.println("Random Nonce    : " +

            Base64.getEncoder().encodeToString(nonce));

    }

}
```

OUTPUT:

Random Session Key: X8sZrU9wOeYqjvEo7p94QQ==

Random Nonce : afm9dJt2b0E=

Date: 28/11/2025**Experiment:5A****Write a program to generate public/private keys using RSA.****PROGRAM:**

```
import java.security.*;
import java.util.Base64;

public class RSA_KeyGen {

    public static void main(String[] args) throws Exception {

        // Create RSA key pair

        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");

        keyGen.initialize(2048); // Key size

        KeyPair pair = keyGen.generateKeyPair();

        PublicKey pubKey = pair.getPublic();

        PrivateKey privKey = pair.getPrivate();

        // Print keys in Base64

        System.out.println("Public Key:\n" +

            Base64.getEncoder().encodeToString(pubKey.getEncoded()));

        System.out.println("\nPrivate Key:\n" +

            Base64.getEncoder().encodeToString(privKey.getEncoded()));

    }

}
```

OUTPUT:

Public Key:

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEApQZ5uUT9...

Private Key:

MIIEvQIBADANBgkqhkiG9w0BAQEFAASCBAKcwggSjAgEAAoIBAQC...

Date: 28/11/2025	Encrypt and decrypt a message with generated keys.
Experiment:5B	

PROGRAM:

```
import javax.crypto.Cipher;
```

```
import java.security.*;
```

```
import java.util.Base64;
```

```
public class RSA_EncryptDecrypt {
```

```
    public static void main(String[] args) throws Exception {
```

```
        // 1. Generate RSA keys
```

```
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
```

```
        keyGen.initialize(2048);
```

```
        KeyPair pair = keyGen.generateKeyPair();
```

```
        PublicKey pubKey = pair.getPublic();
```

```
        PrivateKey privKey = pair.getPrivate();
```

```
        String message = "Hello RSA";
```

```
        // 2. Encrypt using Public Key
```

```
        Cipher encryptCipher = Cipher.getInstance("RSA");
```

```
        encryptCipher.init(Cipher.ENCRYPT_MODE, pubKey);
```

```
        byte[] encryptedBytes = encryptCipher.doFinal(message.getBytes());
```

```
        String encryptedText = Base64.getEncoder().encodeToString(encryptedBytes);
```

```
        // 3. Decrypt using Private Key
```

```
        Cipher decryptCipher = Cipher.getInstance("RSA");
```

```
decryptCipher.init(Cipher.DECRYPT_MODE, privKey);

String decryptedText = new String(
    decryptCipher.doFinal(Base64.getDecoder().decode(encryptedText)));

// 4. Output

System.out.println("Original Message : " + message);
System.out.println("Encrypted Message: " + encryptedText);
System.out.println("Decrypted Message: " + decryptedText);
}
}
```

OUTPUT:

Original Message : Hello RSA

Encrypted Message: MLdZ1H1Gm97oP8xT0a5Qn2dLhD6S3d...

Decrypted Message: Hello RSA

Date: 05/12/2025	Implement SHA-512 hashing of a message using a programming library.
Experiment:6A	

PROGRAM:

```
import java.security.MessageDigest;
import java.util.Base64;

public class SHA512_Hash {
    public static void main(String[] args) throws Exception {
        String message = "Hello SHA-512";
        // Create SHA-512 digest object
        MessageDigest md = MessageDigest.getInstance("SHA-512");
        // Compute hash
        byte[] hashBytes = md.digest(message.getBytes());
        // Convert to Base64 for readable output
        String hashValue = Base64.getEncoder().encodeToString(hashBytes);
        System.out.println("Message : " + message);
        System.out.println("SHA-512 Hash : " + hashValue);
    }
}
```

OUTPUT:

Message : Hello SHA-512

SHA-512 Hash :

ySb6S4u2D9xZK2lxYbP8m5rX6o8kC0jHjEf8lJv0t5vE3Tkb6ty4pYV1OON+1xg6
gW8G4Zl8EQcVjL6u9zttw==

Date: 05/12/2025**Experiment:6B****Generate and verify HMAC for a given message and key.****PROGRAM:**

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.util.Base64;

public class HMAC_Demo {

    // Method to generate HMAC

    public static String generateHMAC(String message, String key) throws Exception {

        Mac mac = Mac.getInstance("HmacSHA256");

        SecretKeySpec sk = new SecretKeySpec(key.getBytes(), "HmacSHA256");

        mac.init(sk);

        return Base64.getEncoder().encodeToString(mac.doFinal(message.getBytes()));

    }

    public static void main(String[] args) throws Exception {

        String message = "This is HMAC test";

        String key = "mysecretkey";

        // Generate HMAC

        String hmacValue = generateHMAC(message, key);

        // Verify HMAC

        String recomputed = generateHMAC(message, key);

        boolean isValid = hmacValue.equals(recomputed);

        System.out.println("Message    : " + message);

        System.out.println("Secret Key : " + key);

    }

}
```

```
System.out.println("Generated HMAC: " + hmacValue);  
System.out.println("Is HMAC Valid? : " + isValid);  
}  
}
```

OUTPUT:

Message : This is HMAC test

Secret Key : mysecretkey

Generated HMAC: 1gVja5xwqfxs4Eep5gT6xZmxhqL3fiNjM8d3sYrBf3o=

Is HMAC Valid? : true

Date: 19/12/2025	Use RSA or DSA to generate a digital signature for a message.
Experiment:7A	

PROGRAM:

```
import java.security.*;
import java.util.Base64;

public class DigitalSignature_Generate {

    public static void main(String[] args) throws Exception {

        String message = "Hello Digital Signature";

        // Generate RSA Key Pair

        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");

        keyGen.initialize(2048);

        KeyPair pair = keyGen.generateKeyPair();

        PrivateKey privateKey = pair.getPrivate();

        PublicKey publicKey = pair.getPublic();

        // Create Signature Object

        Signature sign = Signature.getInstance("SHA256withRSA");

        sign.initSign(privateKey);

        sign.update(message.getBytes());

        // Generate Digital Signature

        byte[] signatureBytes = sign.sign();

        String signature = Base64.getEncoder().encodeToString(signatureBytes);

        System.out.println("Message: " + message);

        System.out.println("Digital Signature:\n" + signature);

    }

}
```

```
// Print Public Key for verification step

System.out.println("\nPublic Key (Base64):\n" +

    Base64.getEncoder().encodeToString(publicKey.getEncoded()));

}

}
```

OUTPUT:

Message: Hello Digital Signature

Digital Signature:

U3Y4g5VJ6gWiEfK5Ytyl2H0V9yZ7EJqU8JQ0BvJx1bW7U0nZC8q6...

Public Key (Base64):

MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA v6l0...

Date: 19/12/2025	Verify the signature at the receiver side.
Experiment:7B	

PROGRAM:

```
import java.security.*;
import java.util.Base64;

public class DigitalSignature_Verify {
    public static void main(String[] args) throws Exception {
        String message = "Hello Digital Signature";

        // Generate RSA Key Pair (sender keys)
        KeyPairGenerator keyGen = KeyPairGenerator.getInstance("RSA");
        keyGen.initialize(2048);
        KeyPair pair = keyGen.generateKeyPair();

        PrivateKey privateKey = pair.getPrivate();
        PublicKey publicKey = pair.getPublic();

        // Sender generates signature
        Signature sign = Signature.getInstance("SHA256withRSA");
        sign.initSign(privateKey);
        sign.update(message.getBytes());
        String signature = Base64.getEncoder().encodeToString(sign.sign());

        // Receiver verifies signature
        Signature verifySig = Signature.getInstance("SHA256withRSA");
        verifySig.initVerify(publicKey);
        verifySig.update(message.getBytes());

        boolean isValid = verifySig.verify(Base64.getDecoder().decode(signature));

        System.out.println("Received Message : " + message);
        System.out.println("Received Signature: " + signature);
        System.out.println("Is Signature Valid? : " + isValid);
    }
}
```

OUTPUT:

Received Message : Hello Digital Signature

Received Signature: T3c7dOvl3RkZTm1t0x2JtVgrEX0P4C1m...

Is Signature Valid? : true

Date: 02/01/2026	Simulate a simple authentication protocol between client and server using shared keys.
Experiment:8A	

PROGRAM:

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.security.SecureRandom;
import java.util.Base64;

public class SimpleAuth_Task1 {
    // Method to generate HMAC
    public static String hmac(String data, String key) throws Exception {
        Mac mac = Mac.getInstance("HmacSHA256");
        SecretKeySpec sk = new SecretKeySpec(key.getBytes(), "HmacSHA256");
        mac.init(sk);
        return Base64.getEncoder().encodeToString(mac.doFinal(data.getBytes()));
    }

    public static void main(String[] args) throws Exception {
        String sharedKey = "mysecretkey"; // Pre-shared key
        // Step 1: Server generates random challenge
        SecureRandom sr = new SecureRandom();
        String challenge = "CHALLENGE_" + sr.nextInt(9999999);
        System.out.println("Server → Client: " + challenge);
        // Step 2: Client generates HMAC(response)
        String clientResponse = hmac(challenge, sharedKey);
        System.out.println("Client → Server: " + clientResponse);
    }
}
```

```
// Step 3: Server verifies  
  
String expected = hmac(challenge, sharedKey);  
  
boolean isValid = clientResponse.equals(expected);  
  
System.out.println("\nAuthentication Successful? : " + isValid);  
}  
}
```

OUTPUT:

Server → Client: CHALLENGE_529384

Client → Server: 8hYx4wmaVqOivbYH3bgT0fthj8c8bLp2Yp8D6W9VaZk=

Authentication Successful? : true

Date: 02/01/2026	Demonstrate mutual authentication process.
Experiment:8B	

PROGRAM:

```
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;
import java.security.SecureRandom;
import java.util.Base64;

public class MutualAuth_Task2 {

    public static String hmac(String data, String key) throws Exception {

        Mac mac = Mac.getInstance("HmacSHA256");
        mac.init(new SecretKeySpec(key.getBytes(), "HmacSHA256"));
        return Base64.getEncoder().encodeToString(mac.doFinal(data.getBytes()));
    }

    public static void main(String[] args) throws Exception {

        String sharedKey = "sharedsecretkey";

        SecureRandom sr = new SecureRandom();

        // -----
        // Step 1: Server authenticates Client
        // -----

        String challengeToClient = "S_CHALLENGE_" + sr.nextInt(1000000);

        System.out.println("Server → Client: " + challengeToClient);

        String clientResponse = hmac(challengeToClient, sharedKey);

        System.out.println("Client → Server: " + clientResponse);
    }
}
```

```
boolean clientValid = clientResponse.equals(hmac(challengeToClient, sharedKey));

System.out.println("Is Client Valid? : " + clientValid);

// -----

// Step 2: Client authenticates Server

// -----

String challengeToServer = "C_CHALLENGE_" + sr.nextInt(1000000);

System.out.println("\nClient → Server: " + challengeToServer);

String serverResponse = hmac(challengeToServer, sharedKey);

System.out.println("Server → Client: " + serverResponse);

boolean serverValid = serverResponse.equals(hmac(challengeToServer, sharedKey));

System.out.println("Is Server Valid? : " + serverValid);

// -----

// Final result

// -----

if (clientValid && serverValid)

    System.out.println("\n🔒 Mutual Authentication Successful!");

else

    System.out.println("\n❌ Mutual Authentication Failed!");

}

}
```

OUTPUT:

Server → Client: S_CHALLENGE_825193

Client → Server: d8bN4WmO8xlQjdlbWc5z5okbHRdmCVFUZy1HU1Pgx2s=

Is Client Valid? : true

Client → Server: C_CHALLENGE_429510

Server → Client: h08DMfbQwy0nxuDn3c+XzG4M880H5GLdYuh9CYwsS1s=

Is Server Valid? : true

 Mutual Authentication Successful!

Date: 09/01/2026	Setup a simple HTTPS server and client using Open SSL.
Experiment:9A	

PROCEDURE:**STEP1: Install OpenSSL**

- `sudo apt update`
- `sudo apt install openssl`

STEP2: Generate RSA Private Key

- `openssl genrsa -out server.key 2048`

STEP3: Generate Self-Signed Certificate

- `openssl req -new -x509 -key server.key -out server.crt -days 365`
- **You will be asked questions:**
Country Name (2 letter code) [AU]: IN
State or Province Name [Some-State]: Telangana
Locality Name []: Hyderabad
Organization Name []: Student
Organizational Unit Name []: Department
Common Name []: localhost
Email Address []: student@gmail.com
- For **Common Name**, ALWAYS use **localhost** when testing locally.

STEP4: Start HTTPS Server using OpenSSL.

- `openssl s_server -key server.key -cert server.crt -accept 4433 -www`
- This starts a **TLS-enabled server** on port **4433**.

STEP5: Connect Using HTTPS Client (OpenSSL)

- Open another terminal and run:
- `openssl s_client -connect localhost:4433`

OUTPUT:

```

root@kali:~#
root@kali:~# sudo apt update
Get:1 http://mirrors.esto.network/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 Packages [20.6 MB]
Get:3 http://mirrors.esto.network/kali kali-rolling/main amd64 Contents (deb) [52.0 MB]
Get:4 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Packages [117 kB]
Get:5 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Contents (deb) [268 kB]
Get:6 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:7 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Contents (deb) [890 kB]
Fetched 74.1 MB in 26s (2,795 kB/s)
1464 packages can be upgraded. Run 'apt list --upgradable' to see them.

root@kali:~# openssl genrsa -out server.key 2048

root@kali:~# openssl req -new -x509 -key server.key -out server.crt -days 365
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

.
Country Name (2 letter code) [AU]:IN
State or Province Name (full name) [Some-State]:Telangana
Locality Name (eg, city) []:Hyderabad
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Student
Organizational Unit Name (eg, section) []:Department
Common Name (e.g. server FQDN or YOUR name) []:localhost
Email Address []:student@gmail.com

root@kali:~# openssl s_server -key server.key -cert server.crt -accept 4433
Using default temp DH parameters
ACCEPT
hi
dfdf
hello

```

```

root@kali:~# openssl s_server -key server.key -cert server.crt -accept 4433 -www
Using default temp DH parameters
ACCEPT
hi
dfdf
hello

```

```

Session Actions Edit View Help
root@kali:~#
root@kali:~# sudo apt update
Get:1 http://mirrors.esto.network/kali kali-rolling InRelease [34.0 kB]
Get:2 http://mirrors.esto.network/kali kali-rolling/main amd64 Packages [20.6 MB]
Get:3 http://mirrors.esto.network/kali kali-rolling/main amd64 Contents (deb) [52.0 MB]
Get:4 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Packages [117 kB]
Get:5 http://mirrors.esto.network/kali kali-rolling/contrib amd64 Contents (deb) [268 kB]
Get:6 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Packages [188 kB]
Get:7 http://mirrors.esto.network/kali kali-rolling/non-free amd64 Contents (deb) [890 kB]
Fetched 74.1 MB in 26s (2,795 kB/s)
1464 packages can be upgraded. Run 'apt list --upgradable' to see them.

root@kali:~# openssl genrsa -out server.key 2048

root@kali:~# openssl req -new -x509 -key server.key -out server.crt -days 365
You are about to be asked to enter information that will be incorporated
into your certificate request.
What you are about to enter is what is called a Distinguished Name or a DN.
There are quite a few fields but you can leave some blank
For some fields there will be a default value,
If you enter '.', the field will be left blank.

.
Country Name (2 letter code) [AU]:IN
State or Province Name (full name) [Some-State]:Telangana
Locality Name (eg, city) []:Hyderabad
Organization Name (eg, company) [Internet Widgits Pty Ltd]:Student
Organizational Unit Name (eg, section) []:Department
Common Name (e.g. server FQDN or YOUR name) []:localhost
Email Address []:student@gmail.com

```

Date: 09/01/2026

Capture and inspect TLS handshake messages using Wire shark.

Experiment:9B

PROCEDURE:**Step 1: Install Wireshark**

- sudo apt install wireshark.

Step 2: Start Wireshark

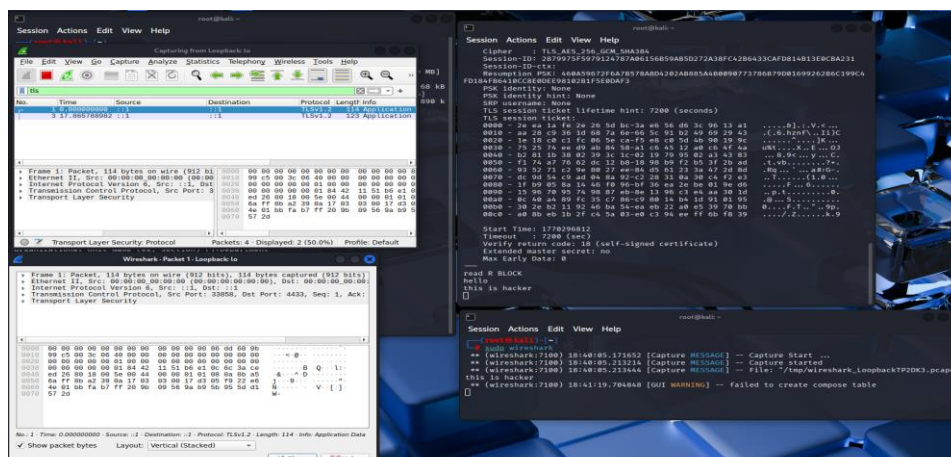
- Sudo wireshark.
 - lo (loopback) — because HTTPS server is running on localhost.

Step 3: Apply TLS Filter

- In the top filter bar type:
TYPE: tls

Step 4: Inspect Handshake Messages

- Click **Client Hello**:
 - You will see:
 - TLS Version: **1.3**
 - Random Data
 - Cipher Suites
 - Extensions
- Click **Server Hello**:
 - Selected Cipher
 - Certificate details



Date: 23/01/2026	Configure basic firewall rules using iptables or Windows firewall.
Experiment:10A	

PROCEDURE:**Step 1:** Install required packages

- `sudo apt install iptables iptables-persistent nmap snort -y`

Step 2: Flush existing rules (fresh start)

- `sudo iptables -F`
- `sudo iptables -X`
- `sudo iptables -Z`

Step 3: Set default policies (secure baseline)

- `sudo iptables -P INPUT DROP`
- `sudo iptables -P FORWARD DROP`
- `sudo iptables -P OUTPUT ACCEPT`

Step 4: Allow loopback traffic (required for system)

- `sudo iptables -A INPUT -i lo -j ACCEPT`

Step 5: Allow established connections

- `sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT`

Step 6: Allow SSH access (Port 22)

- `sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT`

Step 7: Allow Port 8443 (Custom Web Panel / Application)

- `sudo iptables -A INPUT -p tcp --dport 8443 -j ACCEPT`

Step 8: Enable logging for dropped packets

- `sudo iptables -A INPUT -j LOG --log-prefix "IPTABLES-DROP: " --log-level 4`
- `sudo journalctl -f`

OUTPUT:

```

(root@kali)-[~]
# sudo apt install iptables iptables-persistent nmap snort -y
iptables is already the newest version (1.8.11-4).
iptables-persistent is already the newest version (1.0.24).
nmap is already the newest version (7.98+dfsg-1kali1).
snort is already the newest version (3.10.2.0-0kali1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1464

(root@kali)-[~]
# sudo iptables -F
sudo iptables -X
sudo iptables -Z

(root@kali)-[~]
# sudo iptables -P INPUT DROP
sudo iptables -P FORWARD DROP
sudo iptables -P OUTPUT ACCEPT

(root@kali)-[~]
# sudo iptables -A INPUT -i lo -j ACCEPT

(root@kali)-[~]
# sudo iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

(root@kali)-[~]
# sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

(root@kali)-[~]
# sudo iptables -A INPUT -p tcp --dport 8443 -j ACCEPT

(root@kali)-[~]
# sudo iptables -A INPUT -j LOG --log-prefix "IPTABLES-DROP: " --log-level 4
sudo iptables -A INPUT -j DROP

(root@kali)-[~]
# sudo netfilter-persistent save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/15-ip4tables save
run-parts: executing /usr/share/netfilter-persistent/plugins.d/25-ip6tables save

(root@kali)-[~]
# sudo journalctl -f
Feb 05 19:10:10 kali dbus-daemon[914]: [session uid=1000 pid=914 pidfd=5] Activating via systemd: se
rvice name='org.xfce.Xfconf' unit='xfconfd.service' requested by ':1.88' (uid=1000 pid=23399 comm="/
usr/libexec/xfce4-screensaver-dialog --monitor=0")
Feb 05 19:10:10 kali systemd[891]: Starting xfconfd.service - Xfce configuration service ...
Feb 05 19:10:10 kali dbus-daemon[914]: [session uid=1000 pid=914 pidfd=5] Successfully activated ser
vice 'org.xfce.Xfconf'
Feb 05 19:10:10 kali systemd[891]: Started xfconfd.service - Xfce configuration service.
Feb 05 19:10:14 kali xfce4-screensaver-dialog[23399]: gkr-pam: unlocked login keyring
Feb 05 19:10:14 kali xfce4-screensaver-dialog[23471]: pam_unix(xfce4-screensaver:account): setuid fa
iled: Operation not permitted
Feb 05 19:10:31 kali rtkit-daemon[792]: Supervising 7 threads of 4 processes of 1 users.
Feb 05 19:10:31 kali rtkit-daemon[792]: Supervising 7 threads of 4 processes of 1 users.
Feb 05 19:10:51 kali sudo[23794]:      root : TTY=pts/0 ; PWD=/root ; USER=root ; COMMAND=/usr/bin/jo
urnalctl -f
Feb 05 19:10:51 kali sudo[23794]: pam_unix(sudo:session): session opened for user root(uid=0) by rak

```


Date: 23/01/2026

Generate network attacks (e.g., port scan using Nmap) and observe firewall logs.

Experiment:10B

PROCEDURE:

- sudo apt install nmap

Step 1: Perform a Normal Scan

- nmap <target-ip>

Step 2: Perform a Stealth SYN Scan

- sudo nmap -sS <target-ip>

Step 3: Enable Logging in IPTABLES

- sudo iptables -A INPUT -j LOG --log-prefix "IPTables-Dropped: "
- Logs will appear in:
 - { /var/log/syslog }
- View Logs
 - sudo tail -f /var/log/syslog

```
(root@kali)~#
# sudo nmap -sS 10.0.2.15
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-05 19:17 +0530
Stats: 0:00:48 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 23.00% done; ETC: 19:20 (0:02:37 remaining)
Stats: 0:01:00 elapsed; 0 hosts completed (1 up), 1 undergoing SYN Stealth Scan
SYN Stealth Scan Timing: About 29.00% done; ETC: 19:20 (0:02:24 remaining)

(root@kali)~#
# sudo nmap -sS 10.2.0.15
Starting Nmap 7.98 ( https://nmap.org ) at 2026-02-05 19:18 +0530
Nmap scan report for 10.2.0.15
Host is up (0.0013s latency).
All 1000 scanned ports on 10.2.0.15 are in ignored states.
Not shown: 1000 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 5.06 seconds

(root@kali)~#
# sudo iptables -A INPUT -j LOG --log-prefix "IPTables-Dropped: "

(root@kali)~#
# sudo journalctl -k | grep IPTABLES
Feb 05 18:54:06 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=142.250.77.129 DST=10.0.2.15 LEN=113 TOS=0x00 PREC=0x00 TTL=64 ID=65137 PROTO=TCP SPT=443 DPT=55878 WINDOW=65535 RES=0x00 ACK PSH FIN URG=0
Feb 05 18:54:06 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=142.251.221.99 DST=10.0.2.15 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=65138 PROTO=TCP SPT=443 DPT=38742 WINDOW=65535 RES=0x00 ACK FIN URG=0
Feb 05 18:54:06 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=142.250.182.69 DST=10.0.2.15 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=65139 PROTO=TCP SPT=443 DPT=39718 WINDOW=65535 RES=0x00 ACK FIN URG=0
Feb 05 18:54:06 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=172.217.24.106 DST=10.0.2.15 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=65140 PROTO=TCP SPT=443 DPT=43396 WINDOW=65535 RES=0x00 ACK FIN URG=0
Feb 05 18:54:06 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=142.250.67.78 DST=10.0.2.15 LEN=40 TOS=0x00 PREC=0x00 TTL=64 ID=65141 PROTO=TCP SPT=443 DPT=3534 WINDOW=65535 RES=0x00 ACK FIN URG=0
Feb 05 18:54:08 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=172.217.24.97 DST=10.0.2.15 LEN=208 TOS=0x00 PREC=0x00 TTL=64 ID=65143 PROTO=TCP SPT=443 DPT=34036 WINDOW=65535 RES=0x00 ACK PSH FIN URG=0
Feb 05 18:54:08 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=142.250.205.99 DST=10.0.2.15 LEN=208 TOS=0x00 PREC=0x00 TTL=64 ID=65144 PROTO=TCP SPT=443 DPT=39774 WINDOW=65535 RES=0x00 ACK PSH FIN URG=0
Feb 05 18:54:08 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=172.217.26.42 DST=10.0.2.15 LEN=208 TOS=0x00 PREC=0x00 TTL=64 ID=65145 PROTO=TCP SPT=443 DPT=46548 WINDOW=65535 RES=0x00 ACK PSH FIN URG=0
Feb 05 18:54:08 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=142.250.67.78 DST=10.0.2.15 LEN=220 TOS=0x00 PREC=0x00 TTL=64 ID=65146 PROTO=TCP SPT=443 DPT=35546 WINDOW=65535 RES=0x00 ACK PSH FIN URG=0
Feb 05 18:54:08 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=142.250.205.99 DST=10.0.2.15 LEN=220 TOS=0x00 PREC=0x00 TTL=64 ID=65142 PROTO=TCP SPT=443 DPT=35912 WINDOW=65535 RES=0x00 ACK URG=0
Feb 05 18:54:08 kali kernel: IPTABLES-DROP: IN=eth0 OUT= MAC=08:00:27:cf:58:20:52:55:0a:00:02:02:08:00 SRC=172.217.26.42 DST=10.0.2.15 LEN=169 TOS=0x00 PREC=0x00 TTL=64 ID=65147 PROTO=TCP SPT=443 DPT=
```

Date: 23/01/2026	Analyze IDS alerts using Snort (or Suricata).
Experiment:10C	

PROCEDURE:**Step 1: Install Snort**

- `sudo apt install snort -y`

Step 2: Run Snort in IDS Mode

- `sudo snort -i eth0 -A console`

For localhost traffic:

- `sudo snort -i lo -A console`

Step 3: Add Custom Snort Rule

- `sudo nano /etc/snort/rules/local.rules`

Add rule:

- `alert tcp any any -> any 80 (msg:"HTTP Access Detected"; sid:1000001;)`

Step 4: Run Snort with Config

- `sudo snort -A console -q -c /etc/snort/snort.conf -i eth0`

Step 5: Generate Traffic

- **Open Powershell** { `curl http://<target-ip> }`

Snort output:

- `[**] [1:1000001:0] HTTP Access Detected [**]`

OUTPUTS:

```
+ sudo snort -c /etc/snort/snort.lua -i lo -o eth0
[...]  
")~ Snort++ 3.10.2.0 on localhost (127.0.0.1) are in  
loading /etc/snort/snort.lua:  
loading snort_defaults.lua: next up! scanned in 0.00 seconds  
finished snort_defaults.lua:  
output /home/rakesh  
stream_ip 10.0.0.15  
stream_icmp connect to 10.0.0.15 port 80 with  
stream_tcp  
stream_udp /home/rakesh  
stream_user  
stream_file  
arp_spoof  
back_orifice  
imap  
netflow  
normalizer  
pop  
rpc_decode  
sip  
ssh  
ssl  
telnet  
cip  
dnp3  
iec104  
mms  
modbus  
opcua  
s7commplus  
dce_smb  
dce_tcp  
dce_udp  
dce_http_proxy  
dce_http_server  
gtp_inspect  
port_scan  
smtp  
ftp_server  
ftp_client  
http_inspect  
http2_inspect  
js_norm  
appid  
wizard  
binder  
active  
alerts  
daq  
decode  
host_cache  
host_tracker  
hosts
```

```

network
Finished /etc/snort/snort.lua: 0.00 seconds
Loading file_id.rules_file: 0.00 seconds
Loading file_magic.rules: 0.00 seconds
Finished file_magic.rules: 0.00 seconds
Finished file_id.rules_file: 0.00 seconds

ips policies rule stats
      id loaded shared enabled file
      0 219 0 0 219 /etc/snort/snort.lua

rule counts
total rules loaded: 219
    text rules: 219
    option chains: 219
    chain headers: 1

service rule counts
      file_id: 219 219
      total: 219 219

fast pattern groups
      to_server: 1
      to_client: 1

search engine (ac_bnfa)
      instances: 2
      patterns: 438
      pattern chars: 2602
      num states: 1832
      num match states: 392
      memory scale: KB
      total memory: 71.2812
      pattern memory: 19.6484
      match list memory: 28.4375
      transition memory: 22.9453
appid: MaxRss diff: 3072
appid: patterns loaded: 300

pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 95 ms
++ [0] lo

```